MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Petcov Nicolai FAF-233

Report

Laboratory Work No. 6

Information Security and Cryptography

Hash Functions and Digital Signatures

Checked by:
Zaica M., *university assistant*
DISA, FCIM, UTM

Chișinău – 2025

1 Objective

Study and implement cryptographic hash functions and digital signature algorithms. Understand the principles of message authentication, integrity verification, and non-repudiation. Develop practical skills in implementing RSA and ElGamal digital signatures with modern hash functions.

2 Tasks

1. Implement RSA digital signature:
   - Generate RSA keys with modulus n of at least 3072 bits
   - Select hash function using formula: $i = (k \bmod 24) + 1$
   - Sign message from Laboratory Work No. 2
   - Verify the digital signature
   - Convert hash to decimal representation
2. Implement ElGamal digital signature:
   - Use provided parameters: 2048-bit prime p and generator $g = 2$
   - Select hash function using formula: $i = (k \bmod 24) + 1$ (different list)
   - Sign message from Laboratory Work No. 2
   - Verify the digital signature
   - Convert hash to decimal representation
3. Provide detailed step-by-step explanations of all algorithms
4. Demonstrate signature invalidation when message is modified

3 Theoretical Background

3.1 Cryptographic Hash Functions

A **cryptographic hash function** H maps data of arbitrary size to a fixed-size bit string:

$$H : \{0,1\}^* \to \{0,1\}^n \tag{1}$$

Essential Properties:

- **Pre-image Resistance:** Given h, infeasible to find m where $H(m) = h$
- **Collision Resistance:** Infeasible to find $m_1 \neq m_2$ with $H(m_1) = H(m_2)$
- **Avalanche Effect:** 1-bit input change affects 50% of output bits

Common Algorithms:

- **MD5/SHA-1:** BROKEN/DEPRECATED
- **SHA-256, SHA-512:** Widely used
- **SHA-3:** Modern NIST standard

3.2 Digital Signatures

A **digital signature** provides:

- **Authentication:** Verifies sender identity
- **Integrity:** Detects message modification
- **Non-repudiation:** Signer cannot deny signing

Process: Sign: $s = \text{Sign}(H(m), \text{private\_key})$ → Verify: $\text{Verify}(H(m), s, \text{public\_key})$

3.3 RSA Digital Signature

Key Generation:

1. Generate primes p, q → compute $n = p \times q$
2. Compute $\phi(n) = (p-1)(q-1)$
3. Choose $e = 65537$, compute $d = e^{-1} \bmod \phi(n)$

Signing: $S = H(m)^d \bmod n$
Verification: $H'(m) = S^e \bmod n$, valid if $H'(m) = H(m)$
Mathematical Proof:

$$S^e = (H(m)^d)^e = H(m)^{de} \equiv H(m) \pmod{n} \tag{2}$$

(by Euler's theorem, since $de \equiv 1 \pmod{\phi(n)}$)
Security: Based on factorization hardness. Minimum 3072-bit keys (2025).

3.4 ElGamal Digital Signature

Parameters: Prime p, generator g
Key Generation: Choose random x → compute $y = g^x \bmod p$
Signing:

1. Choose random k where $\gcd(k, p-1) = 1$
2. $r = g^k \bmod p$
3. $s = (H(m) - xr) \cdot k^{-1} \bmod (p-1)$
4. Signature: (r, s)

Verification: Check $g^{H(m)} \equiv y^r \cdot r^s \pmod{p}$
Mathematical Proof:

$$y^r \cdot r^s = g^{xr} \cdot g^{ks} = g^{xr+ks} = g^{H(m)} \pmod{p} \tag{3}$$

(since $H(m) = xr + ks \bmod (p-1)$)
Security: Based on discrete log. **CRITICAL:** Never reuse k (PS3 hack 2010).

3.5 Comparison

Property	RSA	ElGamal
Basis	Factorization	Discrete log
Signature size	$\log_2(n)$	$2\log_2(p)$
Deterministic	Yes	No
Randomness	None	Critical
Key size (2025)	3072+ bits	2048+ bits

Table 1: RSA vs ElGamal comparison

4 Implementation

4.1 Hash Function Selection

Both tasks require selecting a hash function based on student's position in class:

```
def get_hash_function(k):
    """
    Determine hash function based on position
    i = (k mod 24) + 1

    k = student number in class list
    i = index in hash function list
    """
    # Task 2 hash list
    hash_functions_task2 = [
        "MD4", "MD5", "MD2", "MD6-128", "MD6-256", "MD6-512",
        "SHA-1", "SHA-224", "SHA-256", "SHA-384", "SHA-512",
        "SHA3-224", "SHA3-256", "SHA3-384", "SHA3-512",
        "RIPEMD-128", "RIPEMD-160", "RIPEMD-256", "RIPEMD-320",
        "Whirlpool", "NTLM", "Haval192,3", "Haval224,4", "Haval256,4"
    ]

    i = (k % 24) + 1
    return hash_functions_task2[i - 1], i
```

Listing 1: Hash Function Selection

4.2 Hash Computation

```
import hashlib

def compute_hash(message, hash_name):
    """
    Compute cryptographic hash of message
    Returns decimal representation
    """
    message_bytes = message.encode('utf-8')

    # Select hash algorithm
    if hash_name == "MD5":
        hash_obj = hashlib.md5(message_bytes)
    elif hash_name == "SHA-256":
        hash_obj = hashlib.sha256(message_bytes)
    elif hash_name == "SHA-512":
        hash_obj = hashlib.sha512(message_bytes)
```

```
    elif hash_name == "SHA3-256":
        hash_obj = hashlib.sha3_256(message_bytes)
    elif hash_name == "SHA3-512":
        hash_obj = hashlib.sha3_512(message_bytes)
    # ... other algorithms

    # Get hexadecimal digest
    hash_hex = hash_obj.hexdigest()

    # Convert to decimal
    hash_decimal = int(hash_hex, 16)

    print(f"Hash (hex): {hash_hex}")
    print(f"Hash (decimal): {hash_decimal}")

    return hash_decimal, hash_hex
```

Listing 2: Computing Message Hash

4.3 Task 2: RSA Digital Signature

4.3.1 Key Generation (3072 bits)

```python
from sympy import isprime, gcd, mod_inverse
import random

def generate_large_prime(bits):
    """Generate prime number of specified bit length"""
    while True:
        candidate = random.getrandbits(bits)
        candidate |= (1 << (bits - 1))  # Ensure high bit
        candidate |= 1                   # Ensure odd

        if isprime(candidate):
            return candidate

def generate_rsa_keys(bits=3072):
    """
    Generate RSA keys with n >= 3072 bits
    """
    half_bits = bits // 2

    # Step 1: Generate p and q
    p = generate_large_prime(half_bits)
    q = generate_large_prime(half_bits)

    # Step 2: Compute n = p * q
    n = p * q

    # Verify bit length
    if n.bit_length() < bits:
        return generate_rsa_keys(bits)

    # Step 3: Compute phi(n) = (p-1)(q-1)
    phi_n = (p - 1) * (q - 1)

    # Step 4: Choose e = 65537
    e = 65537

    # Verify gcd(e, phi(n)) = 1
    if gcd(e, phi_n) != 1:
        return generate_rsa_keys(bits)

    # Step 5: Compute d = e^(-1) mod phi(n)
    d = mod_inverse(e, phi_n)

    # Verify: (d * e) mod phi(n) = 1
    assert (d * e) % phi_n == 1

    return (e, n), (d, n), p, q
```

Listing 3: RSA Key Generation for Signatures

4.3.2 RSA Signing and Verification

```python
def rsa_sign(message_hash, private_key):
    """
    Sign hash using RSA private key

    S = H(m)^d mod n
    """
    d, n = private_key

    # Check hash is smaller than modulus
    if message_hash >= n:
        raise ValueError("Hash too large for modulus")

    # Compute signature
    signature = pow(message_hash, d, n)

    return signature

def rsa_verify(message_hash, signature, public_key):
    """
    Verify RSA signature using public key

    H'(m) = S^e mod n
    Valid if H'(m) == H(m)
    """
    e, n = public_key

    # Compute hash from signature
    computed_hash = pow(signature, e, n)

    # Compare with original hash
    is_valid = (message_hash == computed_hash)

    if is_valid:
        print("    SIGNATURE VALID!")
    else:
        print("    SIGNATURE INVALID!")

    return is_valid
```

Listing 4: RSA Sign and Verify

4.4 Task 3: ElGamal Digital Signature

4.4.1 Key Generation

```python
# Given parameters
P = 323170060713110073001535134778251633 6...  # 2048-bit prime
G = 2

def generate_elgamal_keys(p, g):
    """
    Generate ElGamal keys for signatures
    """
    # Verify p is prime
    assert isprime(p)

    # Step 1: Choose random private key x
    x = random.randint(2, p - 2)

    # Step 2: Compute public key y = g^x mod p
    y = pow(g, x, p)

    return (p, g, y), x
```

Listing 5: ElGamal Key Generation

4.4.2 ElGamal Signing

```python
def elgamal_sign(message_hash, private_key, p, g):
    """
    Sign hash using ElGamal

    1. Choose random k: gcd(k, p-1) = 1
    2. r = g^k mod p
    3. s = (H(m) - x*r) * k^(-1) mod (p-1)

    Returns: (r, s)
    """
    x = private_key

    # Step 1: Find valid k
    for attempt in range(100):
        k = random.randint(2, p - 2)
        if gcd(k, p - 1) == 1:
            break
    else:
        raise ValueError("Could not find valid k")

    # Step 2: Compute r = g^k mod p
    r = pow(g, k, p)

    # Step 3: Compute s = (H(m) - x*r) * k^(-1) mod (p-1)
    k_inv = mod_inverse(k, p - 1)
    h_minus_xr = (message_hash - x * r) % (p - 1)
    s = (h_minus_xr * k_inv) % (p - 1)

    return r, s
```

Listing 6: ElGamal Signature Generation

4.4.3 ElGamal Verification

```python
def elgamal_verify(message_hash, signature, public_key):
    """
    Verify ElGamal signature

    Check: g^H(m) == y^r * r^s (mod p)
    """
    r, s = signature
    p, g, y = public_key

    # Check constraints
    if not (0 < r < p and 0 < s < p - 1):
        print("    SIGNATURE INVALID - constraints violated")
        return False

    # Compute left side: g^H(m) mod p
    left_side = pow(g, message_hash, p)

    # Compute right side: y^r * r^s mod p
    y_r = pow(y, r, p)
    r_s = pow(r, s, p)
    right_side = (y_r * r_s) % p

    # Compare
    is_valid = (left_side == right_side)

    if is_valid:
        print("    SIGNATURE VALID!")
        print("Equation satisfied: g^H(m)     y^r * r^s (mod p)")
    else:
        print("    SIGNATURE INVALID!")
        print("Equation not satisfied")

    return is_valid
```

Listing 7: ElGamal Signature Verification

5 Results and Examples

5.1 RSA Signature Results

Input: Message from Lab 2, $k = 10 \rightarrow$ Hash: SHA-512
Generated Keys: $n = 3072$ bits, $e = 65537$, d computed
Signature: $S = H(m)^d \bmod n$ VALID
Modified Message Test: Different hash $\rightarrow$ INVALID

5.2 ElGamal Signature Results

Input: Same message, p (2048-bit given), $g = 2$
Signature: (r, s) computed with random k
Verification: $g^{H(m)} \equiv y^r \cdot r^s \pmod{p}$ VALID
Modified Message Test: Equation not satisfied $\rightarrow$ INVALID

6 Security Analysis

6.1 Hash Function Security

Recommended (2025): SHA-256, SHA-512, SHA-3
Avoid: MD5 (broken), SHA-1 (deprecated)

6.2 Digital Signature Security

Algorithm	Minimum	Recommended
RSA	3072 bits	4096 bits
ElGamal/DSA	2048 bits	3072 bits
ECDSA	256 bits	384 bits

Table 2: Key size requirements (2025)

Common Vulnerabilities:

- Weak RNG for ElGamal $k \rightarrow$ private key recovery
- Reusing k in ElGamal $\rightarrow$ PS3 hack (2010)
- Small key sizes $\rightarrow$ factorization/discrete log attacks
- Using MD5/SHA-1 $\rightarrow$ collision attacks

Quantum Threat: Shor's algorithm breaks both RSA and ElGamal. Post-quantum alternatives: CRYSTALS-Dilithium, FALCON, SPHINCS+.

7 Conclusions

7.1 Main Achievements

1. Successfully implemented RSA digital signatures with 3072-bit keys
2. Successfully implemented ElGamal digital signatures with given parameters
3. Implemented hash function selection based on student position
4. Demonstrated signature generation and verification for both schemes
5. Validated signature invalidation when message is modified
6. Analyzed security properties of both signature schemes

7.2 Learning Outcomes

Cryptographic Hash Functions:

- Understanding of hash properties (collision resistance, one-way)
- Knowledge of common algorithms (SHA-2, SHA-3)
- Security requirements and attack resistance

Digital Signatures:

- RSA signature scheme and mathematical correctness
- ElGamal signature scheme and verification equation
- Importance of randomness in ElGamal (k must never be reused)
- Purpose: Authentication, integrity, non-repudiation

Security Considerations:

- Key size requirements (RSA 3072+, ElGamal 2048+)
- Vulnerabilities: weak RNG, k reuse, broken hash functions
- Post-quantum threats (Shor's algorithm)

7.3 Practical Implications

Digital signatures are fundamental to modern security:

- TLS/SSL certificates for HTTPS
- Cryptocurrency transactions (Bitcoin, Ethereum)
- Software code signing
- Email security (S/MIME, PGP)
- Legal digital documents

While RSA and ElGamal face quantum computing threats, the principles learned remain foundational to both current and future cryptographic systems including post-quantum alternatives.

Source Code

https://github.com/Nickseen/Cryptology-Security/tree/Lab6